REPORT

Reviewer report: pallets/markupsafe#438: implement

speedups with rust v2

Run markupsafe-438, tested 2026-11-09 10:28 IST by ops/daily_run.py (route source-pair). Proof-of-Change

commit ef12096677e9cf59e8673436d74ec65d53771d7c. Report generated 2026-11-09 12:00 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

COULD NOT VERIFY

Recommended action: Re-run Confidence: Low

The run did not produce a usable comparison, so there is no conclusion about the PR. The run did not finish

cleanly, so it says nothing about the PR.

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: implement speedups with rust v2 (pallets/markupsafe#438).

• What we observed: could not verify: difference(s) no linked claim covers (model-c1-probe c1.import_error), so

the change is not confirmed (26 test(s) ran on main, 26 on the PR; 0 new test(s) pass, 26 unchanged; probe: 1

difference(s) no linked claim covers, shown as advisory only: c1.import_error; probe: 1 observation(s) shown as

advisory only (claim not shown on these inputs; a related change was proven):

c2.bench_contains_rust_speedups (C2); probe: claim(s) C1 proven; probe model-c2-probe has no measured line

coverage (coverage produced no data (exit 0); see probe-coverage.log); it does not count as covering the

change).

• What you should do: Do not decide yet. Fix the problem under 'How to fix' and run the check again.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 9.1 GB (of 15.6 GB).

• Setup check: free disk 92.9 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: markupsafe python setup install instructions https://github.com/pallets/markupsafe site:github.com/pallets/markupsafe

• Query: markupsafe python documentation install https://github.com/pallets/markupsafe site:github.com/pallets/markupsafe

• Query: markupsafe python install requirements https://pypi.org/project/markupsafe/ https://github.com/pallets/markupsafe site:pypi.org

• Source: markupsafe/pyproject.toml at main · pallets/markupsafe · GitHub — https://github.com/pallets/markupsafe/blob/main/pyproject.toml

• Source: markupsafe/pyproject.toml at 4e111271b1494995708a9e878a10829b5e10c7ba · pallets/markupsafe · GitHub — https://github.com/pallets/markupsafe/blob/4e111271b1494995708a9e878a10829b5e10c7ba/pyproject.toml

• Source: MarkupSafe · PyPI — https://pypi.org/project/markupsafe/

• Source: Installing MarkupSafe-2.0.1 on macOS 11.5 · Issue #241 · pallets/markupsafe · GitHub — https://github.com/pallets/markupsafe/issues/241

• Source: Fail to install MarkupSafe · Issue #167 · pallets/markupsafe · GitHub — https://github.com/pallets/markupsafe/issues/167

• Source: "No module named 'markupsafe'" when installing MarkupSafe · Issue #285 · pallets/markupsafe · GitHub — https://github.com/pallets/markupsafe/issues/285

• Setup plan: the PR changes install docs (pyproject.toml, setup.py); setup was read from main's copy and applied the same on both sides.

• Setup plan: the same packages on main and pr.

• Setup plan: install pytest (listed in the tests extra) on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.10, >=3.9

• Installed pytest 9.2.0 from pyproject (listed in the tests extra) on main and pr.

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that

passes on main fails on the PR (26 unchanged, 3 runs per side) | pytest-pr-1.log:7 «26 passed in 0.58s»

| P-C1 | pip install -e . in a virtualenv should build src/markupsafe/_rust_speedups.???.so, assuming you have

Rust installed. (PR description) | proven | probe model-c1-probe, c1.import_success: value: false -> true |

probe-model-c1-probe-pr-1.jsonl:2 «"observation": "c1.import_success"»

| - | Change not in the PR description (probe model-c1-probe) | unmentioned change | c1.import_error: value:

"ImportError" -> null | probe-model-c1-probe-pr-1.jsonl:1 «"observation": "c1.import_error"»

| P-C2 | python bench.py to run all benchmarks, rust included. (PR description) | not observed | probe

model-c2-probe, c2.bench_contains_rust_speedups: same on both sides: false |

probe-model-c2-probe-pr-1.jsonl:1 «"observation": "c2.bench_contains_rust_speedups"»

Citation check (PASS): 4 citation(s), every one re-read from its log file and matched.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 26 / 26 | | Tests unchanged | 26 | | New tests that pass | 0 | | Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | -

| Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 0 | | Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 0 | -

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-c1-probe (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-c1-probe.py, frozen

f6db4c6a672d at 2026-11-09T04:59:13.661097+00:00): Check if rust_speedups module can be imported

• P-C1 (PR description): pip install -e . in a virtualenv should build src/markupsafe/_rust_speedups.???.so,

assuming you have Rust installed. Expects the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c1.import_error | - | differs, not in the PR description | value: "ImportError" -> null |

probe-model-c1-probe-main-1.jsonl:1 «"observation": "c1.import_error"»
probe-model-c1-probe-pr-1.jsonl:1

«"observation": "c1.import_error"»

| c1.import_success | P-C1 | as claimed | value: false -> true | probe-model-c1-probe-main-1.jsonl:2

«"observation": "c1.import_success"»
probe-model-c1-probe-pr-1.jsonl:2 «"observation":

"c1.import_success"»

model-c2-probe (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-c2-probe.py, frozen

2999ea70e5de at 2026-11-09T04:59:13.661298+00:00): Check if bench.py contains reference to rust_speedups

• P-C2 (PR description): python bench.py to run all benchmarks, rust included. Expects the covered observations

to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c2.bench_contains_rust_speedups | P-C2 | no change although the claim says it changes | same on both sides:

false | probe-model-c2-probe-main-1.jsonl:1 «"observation":

"c2.bench_contains_rust_speedups"»
probe-model-c2-probe-pr-1.jsonl:1 «"observation":

"c2.bench_contains_rust_speedups"»

Citation check (PASS): 6 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (1.3 s run).

Coverage gate (python, confidence normal): 1 of 2 changed executable line(s) ran (50%).

| Changed file | Changed executable lines | Of those, ran

| bench.py | 1 | 0

| src/markupsafe/__init__.py | 1 | 1

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested e395cfa3fbca of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: all runs exited 0.

• Changed code was exercised (coverage gate): 1 of 2 changed executable line(s) ran (50%).

• Probes ran on both sides: 2 probe(s), 3 alternating runs per side, every run printed its observations.

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every claim cites a real log line: 4 citation(s), every one re-read from its log file and matched.

• Every probe observation cites its output line: 6 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

| Issue | Status | Detail | How to fix

| Run did not finish | COULD NOT VERIFY | could not verify: difference(s) no linked claim covers

(model-c1-probe c1.import_error), so the change is not confirmed (26 test(s) ran on main, 26 on the PR; 0 new

test(s) pass, 26 unchanged; probe: 1 difference(s) no linked claim covers, shown as advisory only:

c1.import_error; probe: 1 observation(s) shown as advisory only (claim not shown on these inputs; a related

change was proven): c2.bench_contains_rust_speedups (C2); probe: claim(s) C1 proven; probe model-c2-probe

has no measured line coverage (coverage produced no data (exit 0); see probe-coverage.log); it does not count

as covering the change) | Read the setup/build log under Evidence links; build failures usually need a compiler or

a newer nightly.

| Install docs unchanged by the PR | WARN | the PR changes pyproject.toml, setup.py | Setup was read from

main's install docs and applied the same on both sides; the PR's own install steps differ, so read the result with

that in mind.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | all runs exited 0

| Changed code was exercised (coverage gate) | PASS | 1 of 2 changed executable line(s) ran (50%)

| Probes ran on both sides | PASS | 2 probe(s), 3 alternating runs per side, every run printed its observations

| Install docs unchanged by the PR | WARN | the PR changes pyproject.toml, setup.py

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | PASS | 4 citation(s), every one re-read from its log file and matched

| Every probe observation cites its output line | PASS | 6 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).

• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide

REPORT=poc-main/reports/daily/2026-11-08/markupsafe-438 DECISION=approve (or reject,

approve_with_notes, re_run).

Rerun command

make check PR=https://github.com/pallets/markupsafe/pull/438

Evidence links

• Pull request pallets/markupsafe#438

• record.json

• pr.diff

• setup-main.log

• setup-pr.log

• git.log

• pytest-main-1.log

• pytest-pr-1.log

• pytest-main-2.log

• pytest-pr-2.log

• pytest-main-3.log

• pytest-pr-3.log

• junit-main-1.xml

• junit-pr-1.xml

• coverage-pytest.log

• coverage-pr.json

• probe-model-c1-probe-main-1.jsonl

• probe-model-c1-probe-pr-1.jsonl

• probe-model-c2-probe-main-1.jsonl

• probe-model-c2-probe-pr-1.jsonl

• main run 1 result: pytest-main-1.log:7 «26 passed in 1.08s»

• pr run 1 result: pytest-pr-1.log:7 «26 passed in 0.58s»

Glossary

| Term | Meaning

| main | The library as it is today, without the PR.

| PR | Pull request: a proposed change to the library.

| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.

| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-09 12:04 IST